

CAPITAL GAINS • CLI • PYTHON

Refatorando um teste técnico: Lições de Python Avançado

Uma CLI para cálculo de imposto sobre ganhos de capital em operações de renda variável



QUEM SOU EU

Cássio Botaro

Engenheiro de Software · Época Cosméticos

CÓDIGO FONTE

O código está disponível



`github.com/cassiobotaro/capital-gains`

CONTEXTO

O Problema

ENTRADA

```
// stdin – lotes de operações em JSON
[
  { "operation": "buy",   "unit-cost": 10.00, "quantity": 10000 },
  { "operation": "sell",  "unit-cost": 20.00, "quantity":  5000 },
  { "operation": "sell",  "unit-cost":  5.00, "quantity":  5000 }
]
```

SAÍDA

```
// stdout – imposto por operação
[
  { "tax": 0.00    }, // compra → isento
  { "tax": 10000.00 }, // lucro tributável
  { "tax": 0.00    }  // prejuízo
]
```

- **20%**
sobre o lucro acima do preço médio ponderado
- **Isenção**
operações $\leq$ R\$ 20.000,00
- **Prejuízos**
deduzidos de lucros futuros

A Solução

- **Uma CLI que lê lotes de operações via stdin**
Cada linha da entrada é um array JSON de operações independentes, processadas sequencialmente
- **Estado acumulado por lote**
Preço médio ponderado e prejuízo acumulado são mantidos entre operações do mesmo lote
- **Resultado auditável por operação**
Cada operação retorna seu imposto e o novo estado — não apenas o valor final
- **Objetos imutáveis do início ao fim**
Nenhuma variável é mutada — cada passo produz um novo estado a partir do anterior

Fórmula central

```
preço_médio_ponderado_novo = (  
    (quantidade × preço_médio_ponderado) +  
    (quantidade_nova × preço_novo)  
    ) / (quantidade + quantidade_nova)
```

Regra de imposto

```
lucro_líquido = max(0, lucro - prejuízo)  
imposto = 20% × lucro_líquido  
           se volume > R$ 20.000
```


ARQUITETURA

Três módulos, uma narrativa

cli

Leitura de stdin

Parse JSON

Escrita em stdout



money

Objeto de valor imutável

Aritmética monetária

Precisão decimal garantida



tax

Estado do investimento

Regras de negócio

Resultado auditável

CLI — entrada e saída

__MAIN__.PY — PONTO DE ENTRADA

```
import sys
from .cli import process_operations

def main() → None:
    process_operations(sys.stdin, sys.stdout)

# sys.stdin → Iterable[str]
# sys.stdout → IO[str]
```

0 contrato

reader_stream — qualquer `Iterable[str]`

writer_stream — qualquer `IO[str]`

stdin/stdout em produção · StringIO nos testes

CLI.PY — PROCESS_OPERATIONS

```
def process_operations(
    reader_stream: Iterable[str],
    writer_stream: IO[str],
) → None:
    for line in readlines(reader_stream):
        dump_json(
            process_operations_batch(
                parse_json_line(line)
            ),
            writer_stream,
        )
```

↓ expandido nos próximos slides

`readlines()`

`parse_json_line()`

`dump_json()`

CLI — as funções internas

```
def readlines(reader: Iterable[str]) → Iterator[str]:
    for line in reader:
        if line.strip():
            yield line

def parse_json_line(line: str) → list[Operation]:
    raw_ops: list[RawOperation] = json.loads(line)
    return [
        Operation(
            operation=raw["operation"],
            unit_cost=Money(str(raw["unit-cost"])),
            quantity=raw["quantity"],
        )
        for raw in raw_ops
    ]

def dump_json(taxes: list[OperationResult], output: IO[str]) → None:
    formatted = [{"tax": float(r.tax.amount)} for r in taxes]
    json.dump(formatted, output)
    output.write("\n")
```

- 01 **Iterable[str] como parâmetro**
mais simples que IO, aceita stdin, lista ou qualquer iterável
- 02 **Iterator como retorno de readlines**
Iterable → só precisa de `__iter__`;
Iterator → precisa de `__iter__` + `__next__` — mantém estado e só percorre uma vez
- 03 **json.dump vs json.dumps**
dump escreve diretamente no IO stream
— sem string em memória

CLI — TypedDict e deque

TYPEDDICT COM NOTAÇÃO DE FUNÇÃO

```
# chave com hífen – notação de classe
# causaria SyntaxError
RawOperation = TypedDict(
    "RawOperation",
    {
        "operation": Literal["sell", "buy"],
        "unit-cost": float,
        "quantity": int,
    },
)
```

Por que notação de função?

A chave `unit-cost` com hífen é inválida como identificador Python — impossível em sintaxe de classe

TRUQUE COM DEQUE E MAXLEN=0

```
deque(
    map(
        partial(dump_json, output=writer_stream),
        map(
            process_operations_batch,
            map(
                parse_json_line,
                readlines(reader_stream)
            )
        ),
    ),
    maxlen=0, # consome sem criar lista
)
```

Por que maxlen=0?

`deque(iterable, maxlen=0)` consome um iterável lazy sem alocar nada — menor consumo de memória que criar uma lista descartável

Money — estrutura

```
@dataclass(frozen=True)  # gera __init__, __eq__, __repr__
@total_ordering
class Money:
    raw_amount: InitVar[DecimalConvertible]
    amount: Decimal = field(init=False)
    currency: str = "BRL"

    def __post_init__(self, raw_amount):
        # roda após o __init__ gerado pelo dataclass
        quantized = Decimal(raw_amount).quantize(TWOPLACES)
        # frozen=True bloqueia setattr — hack necessário
        object.__setattr__(self, "amount", quantized)

    @classmethod
    def zero(cls, currency: str = DEFAULT_CURRENCY) → "Money":
        return cls("0.00", currency)
```

- 01 @dataclass gera `__init__`, `__eq__`, `__repr__`
sem boilerplate — igualdade por valor e representação legível, automaticamente
- 02 `frozen=True` — objeto imutável
bloqueia mutação após criação — sem efeitos colaterais
- 03 `InitVar` + `field(init=False)`
`raw_amount` existe só no `__init__`; `amount` é sempre 2 casas decimais
- 04 `__post_init__` + `object.__setattr__`
único lugar para converter o valor bruto; contorna o bloqueio do frozen
- 05 `Money.zero()` — construtor alternativo
elimina repetição de `Money("0.00")` — mais expressivo

Money — operações

```
def __add__(self, other: Money) → Money:
    self._assert_same_currency_as(other)
    return Money(self.amount + other.amount)

# __sub__ segue o mesmo padrão de __add__

def __mul__(self, scalar: Scalar) → Money:
    return Money(self.amount * Decimal(scalar))

__rmul__ = __mul__ # Scalar * Money → Money.__rmul__

def __truediv__(self, scalar: Scalar) → Money:
    return Money(self.amount / Decimal(scalar))

def __lt__(self, other: object) → bool:
    if not isinstance(other, Money):
        return NotImplemented
    self._assert_same_currency_as(other)
    return self.amount < other.amount
```

- 01 `__add__` e `__sub__` — Money op Money
retornam um novo objeto — imutabilidade preservada;
operações entre moedas diferentes levantam `ValueError` via `_assert_same_currency_as`
- 02 `__mul__` — Money × Scalar (não Money × Money)
R\$ 10 × 5 faz sentido; R\$ 10 × R\$ 10 não existe — o tipo do parâmetro deixa isso explícito
- 03 `__rmul__ = __mul__` — comutatividade
Python tenta `5 * money` via `int.__mul__` primeiro; ao falhar chama `money.__rmul__` — reutilizar `__mul__` é suficiente
- 04 `__lt__` + `@total_ordering`
implementar só `__lt__` basta — o decorator deriva `__le__`, `__gt__`, `__ge__`; o guard `isinstance` retorna `NotImplemented` para outros tipos

Tax — estrutura

```
@dataclass(frozen=True)
class InvestmentState:
    quantity: int = 0
    weighted_average_price: Money = field(
        default_factory=Money.zero) # acumulador
    accumulated_loss: Money = field(
        default_factory=Money.zero) # acumulador

@dataclass(frozen=True)
class OperationResult:
    new_state: InvestmentState
    tax: Money # toda operação tem imposto (pode ser zero)

def process_operation(
    state: InvestmentState, operation: Operation
) → OperationResult:
    match operation.operation:
        case "buy": return handle_buy(state, operation)
        case "sell": return handle_sell(state, operation)
        case _ as unreachable: assert_never(unreachable)
```

- 01 InvestmentState — acumuladores com default_factory
weighted_average_price e accumulated_loss usam Money.zero como factory — iniciam em zero e acumulam ao longo das operações
- 02 OperationResult — estado + imposto juntos
toda operação retorna o novo estado e o imposto — pode ser zero; permite auditar cada passo, não só o valor final
- 03 Pattern matching — uso literal, não estrutural
Python suporta match com desestruturação de objetos e guards; aqui o uso é o mais simples — equivalente a if/elif; assert_never garante exaustividade em análise estática

Tax — handle_buy

```
def handle_buy(
    state: InvestmentState,
    operation: Operation,
) → OperationResult:
    new_quantity = state.quantity + operation.quantity
    new_wap = (
        (state.weighted_average_price * state.quantity)
        + (operation.unit_cost * operation.quantity)
    ) / (state.quantity + operation.quantity)
    return OperationResult(
        new_state=InvestmentState(
            quantity=new_quantity,
            weighted_average_price=new_wap,
            accumulated_loss=state.accumulated_loss,
        ),
        tax=Money.zero(),
    )
```

01 Compra nunca gera imposto

`tax=Money.zero()` sempre — o imposto só incide em vendas com lucro

02 A fórmula parece comum — mas abstrai Money

toda a aritmética usa o objeto `Money`; multiplicar quantidade (int) por preço (Money) retorna Money — não há escalares soltos na fórmula

03 accumulated_loss é preservado

compras não afetam o prejuízo acumulado — ele passa intacto para o novo estado

04 Estado imutável — novo objeto sempre

`InvestmentState` é frozen; `handle_buy` constrói e retorna um novo estado sem mutar o original

Tax — handle_sell

```
def handle_sell(state, op) → OperationResult:
    def calculate_new_loss(loss, profit):
        return max(Money.zero(), loss - profit)
    def calculate_taxable_profit(loss, profit):
        return max(Money.zero(), profit - loss)
    def calculate_tax(taxable, total):
        if taxable > Money.zero() and total > EXEMPTION_LIMIT:
            return taxable * TAX_RATE
        return Money.zero()

    gross = (op.unit_cost - state.weighted_average_price) * op.quantity
    total = op.unit_cost * op.quantity
    loss = state.accumulated_loss

    return OperationResult(
        new_state=InvestmentState(...,
            accumulated_loss=calculate_new_loss(loss, gross)),
        tax=calculate_tax(
            calculate_taxable_profit(loss, gross), total),
    )
```

01 Funções internas para legibilidade

cada regra de negócio tem um nome; definidas dentro de handle_sell para não poluir o módulo — não são reutilizadas em outro lugar

02 max(Money.zero(), ...) elimina condicionais

uma expressão matemática substitui três if/else — prejuízo e lucro tributável nunca ficam negativos

03 weighted_average_price não muda na venda

o preço médio ponderado só é recalculado em compras — vender não altera a base de custo

Decisões técnicas

01 Objeto Money como ponto de partida

Conhecimento prévio do padrão — garantiu precisão decimal desde o início sem risco de float

02 Imutabilidade em todos os objetos

frozen dataclasses eliminam bugs de estado compartilhado e tornam o fluxo previsível

03 Resultado auditável por operação

Retornar `OperationResult` com `new_state` permite inspecionar cada passo do cálculo, não só o imposto final

04 Iterable como contrato de entrada

Abstração mais simples que IO — facilita testes sem mock de arquivo e aceita qualquer iterável

05 `max(Money.zero(), loss - profit)`

Uma expressão matemática substitui três condicionais — mais legível, sem abrir mão da correção lógica

06 Containerização + pre-commit

Elimina "funciona na minha máquina" — ambiente e estilo são parte do entregável



Demo

```
uv run capital-gains < entrada.txt
```




Dúvidas

capital-gains · Python · objeto dinheiro